

Методические указания к лабораторной работе

"Основы объектно-ориентированного программирования в C++: классы, инкапсуляция, функции-члены"

Цель работы

Изучение принципов объектно-ориентированного программирования в языке C++: создание классов, использование спецификаторов доступа, инкапсуляция данных, разработка функций-членов (включая константные методы), организация взаимодействия объектов, применение контейнеров `vector` и `list`, форматированный ввод/вывод информации.

1. Теоретическая часть

1.1. Структуры как первый шаг к ООП

Структура (`struct`) позволяет объединить несколько переменных разных типов в один тип. Все члены структуры по умолчанию являются открытыми (`public`).

Пример 1. Структура `Point` (точка на плоскости)

```
#include <iostream>
#include <cmath>

struct Point {
    double x { 0.0 };    // инициализаторы членов
    double y { 0.0 };

    // функция-член для вычисления расстояния до другой точки
    double distanceTo(const Point& other) const {
        double dx = x - other.x;
        double dy = y - other.y;
        return std::sqrt(dx*dx + dy*dy);
    }

    // функция-член для вывода координат
    void print() const {
        std::cout << "(" << x << ", " << y << ")";
    }
};

int main() {
    Point p1 { 1.0, 2.0 };    // агрегатная инициализация
    Point p2 { 4.0, 6.0 };
    std::cout << "Расстояние: " << p1.distanceTo(p2) << '\n';
    return 0;
}
```

Пояснения:

- Члены `x` и `y` инициализированы значениями по умолчанию.
- Функция `distanceTo` помечена `const`, так как она не изменяет объект.
- Объекты создаются с помощью агрегатной инициализации (структура остаётся агрегатом).

1.2. Классы и спецификаторы доступа

Класс (`class`) отличается от структуры тем, что его члены по умолчанию являются закрытыми (`private`). Для управления доступом используются спецификаторы `public:` и `private:`.

Пример 2. Класс `BankAccount` (банковский счёт)

```
#include <iostream>
#include <string>

class BankAccount {
private:
    std::string m_owner { "Unknown" };
    double m_balance { 0.0 };

public:
    // установка владельца
    void setOwner(const std::string& owner) {
        m_owner = owner;
    }

    // получение имени владельца (константный метод)
    const std::string& getOwner() const {
        return m_owner;
    }

    // внесение средств
    void deposit(double amount) {
        if (amount > 0)
            m_balance += amount;
    }

    // снятие средств (возвращает true, если операция успешна)
    bool withdraw(double amount) {
        if (amount > 0 && amount <= m_balance) {
            m_balance -= amount;
            return true;
        }
        return false;
    }

    // получение текущего баланса
    double getBalance() const {
        return m_balance;
    }
};
```

```
int main() {
    BankAccount acc;
    acc.setOwner("Иванов Иван");
    acc.deposit(1000);
    acc.withdraw(500);
    std::cout << "Владелец: " << acc.getOwner()
               << ", баланс: " << acc.getBalance() << '\n';
    return 0;
}
```

Пояснения:

- Члены `m_owner` и `m_balance` закрыты – доступ возможен только через методы.
- Методы, не изменяющие объект (например, `getOwner`, `getBalance`), объявлены как `const`.
- Сеттер `setOwner` принимает `const std::string&` для избежания копирования.
- В методах `deposit` и `withdraw` выполняется проверка корректности аргументов (поддержание инварианта: баланс не может стать отрицательным).

1.3. Константные объекты и константные методы

Если объект объявлен как `const`, для него можно вызывать только константные методы. Константный метод гарантирует, что он не изменяет состояние объекта.

Пример 3. Класс `Time` (время)

```
#include <iostream>

class Time {
private:
    int m_hours { 0 };
    int m_minutes { 0 };
    int m_seconds { 0 };

public:
    void setTime(int h, int m, int s) {
        if (h >= 0 && h < 24 && m >= 0 && m < 60 && s >= 0 && s < 60) {
            m_hours = h;
            m_minutes = m;
            m_seconds = s;
        }
    }

    // константный метод – только чтение
    void print() const {
        std::cout << m_hours << ':' << m_minutes << ':' << m_seconds;
    }

    // неконстантный метод – изменяет объект
    void addSeconds(int sec) {
        m_seconds += sec;
    }
}
```

```

        // нормализация (упрощённо)
        m_minutes += m_seconds / 60;
        m_seconds %= 60;
        m_hours += m_minutes / 60;
        m_minutes %= 60;
        m_hours %= 24;
    }

    int getHours() const { return m_hours; }
};

int main() {
    const Time midday { 12, 0, 0 }; // ОШИБКА: класс не агрегат, нужны методы!
    // Правильно:
    Time t;
    t.setTime(12, 0, 0);
    const Time constT = t;    // копия константного объекта
    constT.print();           // ОК
    // constT.addSeconds(10); // Ошибка: неконстантный метод
    return 0;
}

```

Примечание: Объекты классов с закрытыми данными нельзя инициализировать агрегатно. Для создания константного объекта используется уже существующий объект или временный объект (но требуется соответствующий конструктор, который мы пока не рассматриваем). В лабораторной работе мы будем создавать объекты через set-методы, а затем, если нужно, помещать их в контейнер.

1.4. Геттеры и сеттеры. Возврат по ссылке

Геттеры часто возвращают значение по константной ссылке, чтобы избежать копирования. Сеттеры принимают аргументы по значению или по ссылке.

Пример 4. Класс **Person** с именем и возрастом

```

#include <iostream>
#include <string>

class Person {
private:
    std::string m_name { "No name" };
    int m_age { 0 };

public:
    // Сеттер для имени (принимает по значению, можно переместить, но мы просто копируем)
    void setName(const std::string& name) {
        m_name = name;
    }

    // Геттер для имени – возвращает константную ссылку
    const std::string& getName() const {

```

```

        return m_name;
    }

    void setAge(int age) {
        if (age >= 0 && age <= 150)
            m_age = age;
    }

    int getAge() const {
        return m_age;
    }

    // Метод для вывода информации
    void print() const {
        std::cout << "Имя: " << m_name << ", возраст: " << m_age;
    }
};

// Функция, не являющаяся членом, принимает объект по константной ссылке
void celebrateBirthday(Person& p) {
    p.setAge(p.getAge() + 1);    // изменяем объект, поэтому ссылка не const
}

int main() {
    Person p;
    p.setName("Анна");
    p.setAge(25);
    p.print();
    celebrateBirthday(p);
    std::cout << "\nПосле дня рождения: ";
    p.print();
    std::cout << '\n';
    return 0;
}

```

Пояснения:

- Геттер `getName` возвращает `const std::string&`, что позволяет избежать копирования строки.
- Функция `celebrateBirthday` принимает ссылку, чтобы изменить объект.
- Сеттер `setAge` проверяет допустимость значения (инвариант).

1.5. Передача и возврат объектов из функций

Объекты классов можно передавать в функции по значению (создаётся копия), по ссылке (избегаем копирования) или по константной ссылке (если изменения не нужны). Возвращать объекты можно по значению или по (константной) ссылке, но нужно следить за временем жизни.

Пример 5. Возврат ссылки на член данных

```

#include <iostream>
#include <string>

```

```

class Employee {
private:
    std::string m_name;

public:
    void setName(const std::string& name) {
        m_name = name;
    }

    // Возврат константной ссылки на имя
    const std::string& getName() const {
        return m_name;
    }
};

// Функция, возвращающая временный объект Employee
Employee createEmployee(const std::string& name) {
    Employee e;
    e.setName(name);
    return e;
}

int main() {
    Employee john;
    john.setName("John");

    // Используем возвращённую ссылку сразу – OK
    std::cout << john.getName() << '\n';

    // Опасный пример: сохраняем ссылку на член временного объекта
    const std::string& ref = createEmployee("Jane").getName();
    // ref висятякая ссылка, следующий вывод – неопределённое поведение
    // std::cout << ref; // НЕ ДЕЛАТЬ!

    // Безопасно: копируем значение
    std::string nameCopy = createEmployee("Bob").getName();
    std::cout << nameCopy << '\n';
    return 0;
}

```

Предупреждение: Не сохраняйте ссылки на члены временных объектов.

1.6. Контейнеры для хранения объектов: `std::vector` и `std::list`

Стандартные контейнеры позволяют хранить наборы объектов. Для добавления элементов используется `push_back`. Доступ к элементам – через индексы или итераторы.

Пример 6. Хранение банковских счетов в векторе

```

#include <iostream>
#include <vector>
#include <iomanip>

int main() {
    std::vector<BankAccount> accounts;
    int n;
    std::cout << "Введите количество счетов: ";
    std::cin >> n;

    for (int i = 0; i < n; ++i) {
        std::string owner;
        double balance;
        std::cout << "Введите владельца и начальный баланс: ";
        std::cin >> owner >> balance;
        BankAccount acc;
        acc.setOwner(owner);
        acc.deposit(balance);
        accounts.push_back(acc);
    }

    // Вывод в виде таблицы
    std::cout << std::left << std::setw(20) << "Владелец"
              << std::right << std::setw(10) << "Баланс" << '\n';
    for (const auto& acc : accounts) {
        std::cout << std::left << std::setw(20) << acc.getOwner()
                  << std::right << std::setw(10) << std::fixed
                  << std::setprecision(2) << acc.getBalance() << '\n';
    }
    return 0;
}

```

Пояснения:

- Для выравнивания использованы манипуляторы `<iomanip>`: `setw`, `left`, `right`.
- Цикл `for (const auto& acc : accounts)` перебирает элементы по константной ссылке, избегая копирования.

1.7. Инкапсуляция и преимущества скрытия данных

Скрытие данных (инкапсуляция) позволяет:

- Упростить использование класса – пользователь взаимодействует только через открытый интерфейс.
- Поддерживать инварианты класса (например, баланс не может быть отрицательным).
- Легко изменять внутреннюю реализацию без изменения внешнего кода.

Пример 7. Класс `Rectangle` с проверкой инварианта

```
class Rectangle {
private:
    double m_width { 1.0 };
    double m_height { 1.0 };

public:
    void setWidth(double width) {
        if (width > 0) m_width = width;
    }
    void setHeight(double height) {
        if (height > 0) m_height = height;
    }
    double getWidth() const { return m_width; }
    double getHeight() const { return m_height; }
    double area() const { return m_width * m_height; }
};
```

Здесь инвариант: ширина и высота всегда положительны. Прямой доступ к полям нарушил бы этот инвариант.

2. Общее задание

Разработать программу на языке C++, состоящую из нескольких классов (3–4 класса), которые взаимодействуют друг с другом. Программа должна:

1. **Определить классы** в соответствии с вариантом задания.

- Все данные-члены классов должны быть закрытыми (**private**).
- Для доступа к данным использовать открытые методы (геттеры, сеттеры).
- При объявлении членов обязательно использовать **инициализаторы** (например, `int m_id { 0 };`, `std::string m_name { "неизвестно" };`).
- Конструкторы (кроме неявного конструктора по умолчанию) не использовать.
- Динамическое выделение памяти (**new/delete**) не применять.

2. **Организовать ввод данных** с клавиатуры:

- Пользователь вводит количество объектов, затем для каждого объекта вводит соответствующие характеристики.
- Введённые объекты сохраняются в контейнер `std::vector` или `std::list` (выбор обоснован заданием).

3. **Реализовать обработку данных** согласно варианту:

- Вычислительные операции (подсчёт суммы, среднего, поиск максимума/минимума, фильтрация по условию, сортировка и т.п.).
- Для сортировки можно использовать алгоритм `std::sort` из `<algorithm>` с пользовательским компаратором (функцией или лямбда-выражением).
- Для поиска – алгоритм `std::find_if`.

4. Вывести результаты в форматированном виде:

- Использовать таблицы с заголовками и выравниванием столбцов (`<iomanip>`).
- Для вывода вещественных чисел задать точность (`std::fixed` и `std::setprecision`).
- Вывод должен быть понятным и структурированным.

5. Каждый класс должен быть описан в отдельном файле/файлах (либо один заголовочный файл `hpp` или заголовочный файл и файл `cpp`). Код должен быть читаемым, названия переменных и методов – осмысленными.

Требования к реализации классов:

- Каждый класс должен иметь:
 - закрытые поля с инициализаторами;
 - геттеры и сеттеры (где необходимо);
 - методы, реализующие логику, специфичную для класса (например, `deposit`, `withdraw` для банковского счёта);
 - при необходимости константные методы.
 - Для передачи объектов в функции использовать **ссылки** (по возможности константные) – передача по значению допускается только для тривиальных случаев.
 - Не использовать глобальные переменные.
-

3. Варианты заданий

Вариант 1. Библиотека

Классы:

- **Book**: `m_title` (строка), `m_author` (строка), `m_year` (целое), `m_isbn` (строка).
- **Reader**: `m_name` (строка), `m_ticketNumber` (целое).
- **Loan**: `m_reader` (ссылка или указатель? – хранить индекс читателя или сам объект? Лучше хранить копии или использовать идентификаторы. Для простоты будем хранить в **Loan** номера билета читателя и ISBN книги).

Уточнение: Чтобы избежать сложностей с ссылками, будем хранить идентификаторы (номер билета, ISBN) как строки/числа.

Задание:

- Ввести данные о книгах (несколько записей) и читателях.
- Сформировать список выдач: пользователь вводит номер билета и ISBN, книга выдаётся (добавляется запись в вектор `loans` с текущей датой). Дату можно вводить как три числа (день, месяц, год).
- Вывести список должников: те читатели, у которых книга находится больше 30 дней (сравнить с текущей датой, которую тоже ввести).
- Вывести информацию о всех выдачах в виде таблицы.

Вариант 2. Студенты и оценки

Классы:

- **Student**: `m_name`, `m_group` (строка), `m_grades` (вектор целых чисел – оценки).
- **Subject**: `m_name`, `m_hours` (целое).
- **Grade**: `m_studentId` (индекс студента), `m_subjectId` (индекс предмета), `m_value` (целое).

Задание:

- Ввести список студентов и список предметов.
- Ввести оценки (студент, предмет, оценка). Если оценки вводятся несколько раз, можно накапливать в векторе **Grade**.
- Вычислить и вывести:
 - Средний балл каждого студента.
 - Средний балл по каждому предмету.
 - Студента с максимальным средним баллом.
- Вывести таблицу всех оценок (студент, предмет, оценка).

Вариант 3. Банк

Классы:

- **Client**: `m_name`, `m_id` (уникальный номер, целое).
- **Account**: `m_number` (строка), `m_balance` (double), `m_clientId` (целое).
- **Transaction**: `m_accountNumber`, `m_amount` (double), `m_type` (строка "пополнение"/"снятие").

Задание:

- Ввести клиентов и их счета (у одного клиента может быть несколько счетов).
- Выполнить несколько транзакций: пользователь вводит номер счета, сумму и тип. Программа проверяет наличие средств при снятии.
- Вывести выписку по каждому счету: номер, владелец, баланс, список транзакций (если хранить историю, то для каждого счета нужен вектор транзакций). Можно хранить все транзакции в одном векторе и выводить выборкой.

Вариант 4. Спортивная команда

Классы:

- **Player**: `m_name`, `m_age`, `m_position` (строка, например, "нападающий").
- **Team**: `m_name`, `m_players` (вектор индексов игроков или копии? Лучше хранить вектор **Player** непосредственно).
- **Match**: `m_opponent`, `m_teamGoals`, `m_opponentGoals`, `m_date`.

Задание:

- Ввести игроков и сформировать команду (можно просто создать объект **Team** и добавить игроков).
- Провести несколько матчей: ввести результаты.
- Вывести статистику:
 - Общее количество голов команды.
 - Лучшего бомбардира (игрока, забившего больше всего голов) – для этого нужно ввести голы игроков, можно добавить поле `m_goals` в класс **Player** и обновлять его при матчах.

- Таблицу матчей.

Вариант 5. Автосалон

Классы:

- **Car**: `m_brand`, `m_model`, `m_year`, `m_price` (double).
- **Customer**: `m_name`, `m_budget` (double).
- **Sale**: `m_carIndex`, `m_customerIndex`, `m_date`.

Задание:

- Ввести доступные автомобили и покупателей.
- Совершить продажи: пользователь выбирает автомобиль и покупателя, если бюджет покупателя $\geq$ цена автомобиля, продажа регистрируется (создаётся объект **Sale**), бюджет покупателя уменьшается.
- Вывести отчёт:
 - Список проданных автомобилей.
 - Оставшиеся автомобили.
 - Покупателей с их остатком бюджета.

Вариант 6. Зоопарк

Классы:

- **Animal**: `m_species` (вид), `m_name`, `m_age`, `m_cageNumber` (целое).
- **Keeper**: `m_name`, `m_experience` (лет).
- **Feeding**: `m_animalIndex`, `m_keeperIndex`, `m_time` (часы, минуты).

Задание:

- Ввести животных и сотрудников.
- Зарегистрировать кормления (вводятся индексы животного и сотрудника, время).
- Вывести животных, которых не кормили сегодня (сравнить с введённой текущей датой/временем).
- Вывести график кормлений по каждому сотруднику.

Вариант 7. Больница

Классы:

- **Patient**: `m_name`, `m_diagnosis`, `m_room` (целое).
- **Doctor**: `m_name`, `m_specialization`.
- **Appointment**: `m_patientIndex`, `m_doctorIndex`, `m_date` (день, месяц, год).

Задание:

- Ввести пациентов и врачей.
- Записать пациентов на приём (создать записи).
- Вывести расписание приёмов для заданного врача (ввод имени).
- Подсчитать количество пациентов у каждого врача.

Вариант 8. Магазин

Классы:

- **Product**: `m_name`, `m_price` (double), `m_quantity` (целое, остаток).
- **Category**: `m_name` (категория товара).
- **Cart**: `m_productIndex`, `m_quantity` (сколько единиц товара добавлено в корзину).

Задание:

- Ввести товары (с категориями).
- Сформировать корзину покупателя: пользователь выбирает товар и количество, программа проверяет наличие, уменьшает остаток и добавляет позицию в корзину.
- Вывести итоговый чек: список товаров, цена за единицу, количество, стоимость позиции, общая сумма.

Вариант 9. Ресторан

Классы:

- **Dish**: `m_name`, `m_price`, `m_weight` (граммы).
- **Waiter**: `m_name`.
- **Order**: `m_tableNumber` (целое), `m_waiterIndex`, `m_dishes` (вектор индексов блюд), `m_status` (строка "принят", "готов", "оплачен").

Задание:

- Ввести меню (блюда) и официантов.
- Принять заказ: пользователь вводит номер столика, имя официанта, список блюд. Создаётся заказ со статусом "принят".
- По мере готовности менять статус.
- Вывести счёт для столика (сумма всех блюд).
- Вывести выручку по каждому официанту.

Вариант 10. Аэропорт

Классы:

- **Flight**: `m_number` (строка), `m_destination`, `m_departureTime` (часы, минуты).
- **Passenger**: `m_name`, `m_passport`.
- **Ticket**: `m_passengerIndex`, `m_flightIndex`, `m_seat` (строка, например "12A").

Задание:

- Ввести рейсы и пассажиров.
- Продать билеты: пользователь выбирает рейс и пассажира, указывает место. Билет добавляется.
- Вывести список пассажиров на заданный рейс.
- Вывести все рейсы и количество проданных билетов.

Вариант 11. Университет

Классы:

- **Professor**: `m_name`, `m_department`.
- **Course**: `m_name`, `m_credits`, `m_professorIndex`.
- **Enrollment**: `m_studentIndex`, `m_courseIndex`, `m_grade` (целое, 0 – если ещё не оценено).

Задание:

- Ввести профессоров, курсы, студентов (класс **Student** можно взять из варианта 2 или упрощённо: `m_name`, `m_group`).
- Записать студентов на курсы (создать записи **Enrollment**).
- Проставить оценки.
- Вывести для каждого курса среднюю оценку, список студентов с оценками.

Вариант 12. Строительство

Классы:

- **Building**: `m_address`, `m_floors`, `m_material` (строка).
- **Worker**: `m_name`, `m_specialty` (строка).
- **ConstructionSite**: `m_buildingIndex`, `m_workers` (вектор индексов рабочих).

Задание:

- Ввести здания и рабочих.
- Распределить рабочих по объектам (добавить на стройплощадку).
- Вывести список свободных рабочих (не занятых ни на одной стройке).
- Вывести для каждого здания список рабочих.

Вариант 13. Финансы (личный бюджет)

Классы:

- **Category**: `m_name` (например, "еда", "транспорт").
- **Expense**: `m_amount`, `m_date` (день, месяц, год), `m_categoryIndex`.
- **Budget**: `m_month` (целое, номер месяца), `m_limits` (вектор пар: категория, лимит).

Задание:

- Ввести категории расходов.
- Ввести бюджет на месяц (лимиты по категориям).
- Ввести несколько расходов (сумма, дата, категория).
- Вывести отчёт: по каждой категории сумма расходов, превышение лимита (если есть), итого.

Вариант 14. Спортзал

Классы:

- **Member**: `m_name`, `m_membershipUntil` (дата окончания абонеента).
- **Trainer**: `m_name`, `m_specialization`.
- **Workout**: `m_memberIndex`, `m_trainerIndex`, `m_date`, `m_duration` (минуты).

Задание:

- Ввести членов клуба и тренеров.
- Записать тренировки (дата, время, тренер, участник).
- Вывести:
 - Суммарное количество часов тренировок для каждого участника.
 - Самого популярного тренера (по количеству тренировок).
 - Участников с истекшим абонементом (сравнить с текущей датой).

Вариант 15. Туристическое агентство

Классы:

- `Country: m_name, m_capital`.
- `Tour: m_name, m_countryIndex, m_price, m_duration` (дни).
- `Booking: m_touristName, m_tourIndex, m_date`.

Задание:

- Ввести страны и туры.
- Забронировать туры (ввод имени туриста и названия тура).
- Вывести статистику:
 - Самые популярные страны (по количеству бронирований).
 - Общая стоимость всех бронирований.
 - Список туров с указанием свободных мест (можно добавить поле `m_maxParticipants` в класс `Tour` и считать бронирования).

Вариант 16. Кинотеатр

Классы:

- `Movie: m_title, m_genre, m_duration` (минуты).
- `Session: m_movieIndex, m_hall` (целое), `m_time` (часы, минуты).
- `Ticket: m_sessionIndex, m_row, m_seat, m_sold` (bool).

Задание:

- Ввести фильмы и сеансы.
- Продать билеты: пользователь выбирает сеанс, ряд и место, если место свободно – продаётся.
- Вывести схему зала для выбранного сеанса (свободные/занятые места).
- Подсчитать выручку по каждому фильму.

Вариант 17. Музыкальная группа

Классы:

- `Musician: m_name, m_instrument`.
- `Band: m_name, m_musicians` (вектор индексов).
- `Song: m_title, m_duration` (секунды), `m_genre`.

Задание:

- Ввести музыкантов, создать группу (выбрать музыкантов).
- Составить альбом: ввести несколько песен.
- Вывести общую длительность альбома (часы:минуты:секунды).
- Вывести список песен с указанием жанра.

Вариант 18. Книжный магазин

Классы:

- **Author**: `m_name`, `m_country`.
- **Book**: `m_title`, `m_authorIndex`, `m_price`, `m_genre`.
- **Order**: `m_bookIndex`, `m_quantity`, `m_customerName`.

Задание:

- Ввести авторов и книги.
- Оформить заказ: пользователь вводит название книги, количество, имя покупателя. Если книга есть в наличии (можно добавить поле `m_stock` в класс **Book**), заказ добавляется, остаток уменьшается.
- Вывести итоговую стоимость заказа.
- Вывести список книг, отсортированных по цене.

Вариант 19. Ферма

Классы:

- **Animal**: `m_type` (вид), `m_weight`, `m_feedPerDay` (кг корма в день).
- **Feeder** (работник): `m_name`.
- **Feeding**: `m_animalIndex`, `m_feederIndex`, `m_date`, `m_amount` (сколько корма дали).

Задание:

- Ввести животных и работников.
- Зарегистрировать кормления (кто, кого, когда, сколько).
- Подсчитать, сколько корма израсходовано за день.
- Выявить животных, которые получают меньше нормы (сравнить суммарное кормление за день с `m_feedPerDay`).

Вариант 20. Транспортная компания

Классы:

- **Vehicle**: `m_model`, `m_capacity` (грузоподъемность, кг), `m_fuelConsumption` (л/100км).
- **Route**: `m_start`, `m_end`, `m_distance` (км).
- **Trip**: `m_vehicleIndex`, `m_routeIndex`, `m_date`, `m_cargoWeight` (кг).

Задание:

- Ввести транспорт и маршруты.
- Спланировать рейсы: выбрать машину, маршрут, вес груза (не должен превышать грузоподъемность). Рассчитать расход топлива.

- Вывести суммарный расход топлива за все рейсы.
- Вывести список машин, которые использовались больше всего (по количеству рейсов).

Вариант 21. Поликлиника

Классы:

- **Patient**: `m_name`, `m_policyNumber` (строка).
- **Doctor**: `m_name`, `m_room` (кабинет).
- **Prescription**: `m_patientIndex`, `m_doctorIndex`, `m_medication`, `m_date`.

Задание:

- Ввести пациентов и врачей.
- Выписать рецепты (ввод).
- Вывести историю рецептов для заданного пациента.
- Подсчитать количество рецептов, выписанных каждым врачом.

Вариант 22. Гостиница

Классы:

- **Room**: `m_number`, `m_type` ("одноместный", "двухместный"), `m_pricePerNight`, `m_available` (bool).
- **Guest**: `m_name`, `m_passport`.
- **Booking**: `m_guestIndex`, `m_roomIndex`, `m_checkIn`, `m_checkOut` (даты).

Задание:

- Ввести номера и гостей.
- Заселить гостя: выбрать номер на даты, если свободен – создать бронь, номер становится недоступен.
- Выселить (по дате): освободить номер.
- Подсчитать доход за период (по всем бронированиям).

Вариант 23. Фитнес-трекер

Классы:

- **User**: `m_name`, `m_age`, `m_weight`.
- **Activity**: `m_type` ("бег", "ходьба"), `m_date`, `m_duration` (минуты), `m_calories`.
- **Goal**: `m_userIndex`, `m_targetCalories` (цель по калориям в день).

Задание:

- Ввести пользователей и их цели.
- Добавить активности (дата, тип, длительность, рассчитать калории по формуле: например, для бега 10 ккал/мин, для ходьбы 5 ккал/мин – можно зашить в программе).
- Проверить, достиг ли пользователь цели в заданный день.
- Вывести статистику по каждому пользователю: сумма калорий за всё время, средняя длительность активности.

Вариант 24. Интернет-магазин

Классы:

- `Item`: `m_name`, `m_price`, `m_rating` (double, рейтинг).
- `CartItem`: `m_itemIndex`, `m_quantity`.
- `Order`: `m_customerName`, `m_address`, `m_items` (вектор `CartItem`), `m_total`.

Задание:

- Ввести товары.
- Сформировать корзину: пользователь добавляет товары и количество.
- Рассчитать сумму заказа, применить скидку 10%, если сумма > 5000.
- Вывести детали заказа и итоговую сумму.

Вариант 25. Библиотека (усложнённая)

Классы:

- `Library`: `m_name`, `m_address`.
- `Publication`: `m_title`, `m_year`, `m_type` ("книга", "журнал").
- `Borrow`: `m_publicationIndex`, `m_readerName`, `m_borrowDate`, `m_returnDate` (0, если не возвращена).

Задание:

- Ввести публикации (книги и журналы).
- Регистрировать выдачу (читатель, название публикации, дата выдачи).
- Регистрировать возврат (указывается дата возврата).
- Рассчитать штраф за просрочку: если возврат позже чем через 14 дней, штраф 50 руб. за каждый просроченный день.
- Вывести список должников с суммой штрафа.

Примечание по реализации: Для всех вариантов используйте `std::vector` для хранения объектов. Если требуется уникальная идентификация, используйте индексы в векторе (например, `m_readerIndex`). Для работы с датами можно хранить три целых числа (день, месяц, год) или использовать структуру `Date` (но не класс с конструкторами). Для упрощения дату можно вводить как строку "дд.мм.гггг" и разбирать, или просто три числа.

4. Контрольные вопросы

1. Что такое класс? Чем класс отличается от структуры в C++?
2. Какие уровни доступа существуют в C++? Что они означают?
3. Почему рекомендуется делать данные-члены класса закрытыми (private)?
4. Что такое инкапсуляция? Какие преимущества она даёт?
5. Что такое функция-член? Как объявить функцию-член внутри класса?
6. Что такое константная функция-член? Зачем она нужна?
7. Может ли константная функция-член изменять данные объекта? Почему?

8. Можно ли вызывать неконстантную функцию-член для константного объекта?
 9. Что такое геттер и сеттер? Приведите пример.
 10. В каких случаях геттер должен возвращать значение по ссылке, а в каких – по значению?
 11. Почему опасно возвращать неконстантную ссылку на закрытый член?
 12. Что произойдёт, если сохранить ссылку на член временного объекта?
 13. Как передать объект класса в функцию, чтобы избежать копирования?
 14. Чем передача по значению отличается от передачи по ссылке?
 15. Что такое инициализатор члена (member initializer) в определении класса? Приведите пример.
 16. Можно ли инициализировать члены класса при объявлении? Каким синтаксисом?
 17. Почему класс с закрытыми членами не может быть агрегатом?
 18. Как создать объект класса, если у него нет конструкторов? Какие способы инициализации существуют?
 19. Как хранить набор объектов в векторе? Как добавить новый элемент?
 20. Как перебрать элементы вектора с помощью цикла range-based for?
 21. Как отсортировать вектор объектов по определённому полю?
 22. Какие алгоритмы из `<algorithm>` можно использовать для поиска?
 23. Как вывести данные в виде таблицы с выравниванием столбцов?
 24. Для чего нужны манипуляторы `setw`, `left`, `right`?
 25. Что такое инвариант класса? Приведите пример инварианта.
 26. Как сеттер может помочь сохранить инвариант класса?
 27. В чём разница между методами, изменяющими объект, и константными методами?
 28. Можно ли изменить член класса в константном методе, если он объявлен `mutable`? (В лекции не было, но можно упомянуть в вопросе)
 29. Какие функции лучше делать не членами класса, а свободными? Почему?
 30. Каковы преимущества разделения интерфейса и реализации?
 31. Как правильно организовать ввод данных для нескольких объектов?
 32. Что такое «висячая ссылка» и как её избежать?
-

5. Приложение. Справочный материал

5.1. Определение класса

```
class ИмяКласса {  
private:  
    // закрытые данные (доступны только методам класса)  
    тип имя_члена { значение_по_умолчанию };  
public:  
    // открытые методы  
    тип метод(параметры) [const] { тело }  
    // геттеры и сеттеры  
    тип getИмя() const { return m_член; }  
    void setИмя(тип значение) { m_член = значение; }  
};
```

5.2. Спецификаторы доступа

- **private:** – члены доступны только внутри класса.
- **public:** – члены доступны из любого места.
- **protected:** – члены доступны в классе и в производных классах (рассматривается позже).

5.3. Инициализация членов при объявлении

```
class Example {
    int m_id { 0 };           // инициализация значением
    std::string m_name { "неизвестно" };
    double m_coeff { 1.5 };
};
```

Это гарантирует, что при создании объекта поля имеют определённые значения.

5.4. Константные методы

Объявляются добавлением **const** после списка параметров:

```
int getValue() const { return m_value; }
```

Такие методы можно вызывать для константных объектов.

5.5. Передача объектов в функции

- По значению: создаётся копия – дорого для больших объектов.
- По ссылке: **void func(const MyClass& obj)** – копирования нет, объект не изменяется.
- По неконстантной ссылке: **void func(MyClass& obj)** – можно изменять объект.

5.6. Возврат значений из функций-членов

- По значению: **std::string getName() const { return m_name; }** – безопасно, но копирование.
- По константной ссылке: **const std::string& getName() const { return m_name; }** – эффективно, но ссылка должна оставаться действительной (объект должен жить дольше).

5.7. Контейнер **std::vector**

Метод	Описание
push_back(const T& value)	Добавляет элемент в конец
size()	Возвращает количество элементов
operator[](size_t i)	Доступ по индексу (без проверки границ)
at(size_t i)	Доступ с проверкой границ
begin(), end()	Итераторы для перебора

Метод	Описание
<code>clear()</code>	Очищает вектор

Пример перебора:

```
for (size_t i = 0; i < vec.size(); ++i) { vec[i]; }
for (const auto& elem : vec) { /* use elem */ }
```

5.8. Алгоритмы (заголовок `<algorithm>`)

- `std::sort(begin, end, comparator)` – сортировка. Компаратор может быть функцией или лямбдой.

```
std::sort(accounts.begin(), accounts.end(),
    [](const BankAccount& a, const BankAccount& b) {
        return a.getBalance() > b.getBalance();
    });
```

- `std::find_if(begin, end, predicate)` – поиск первого элемента, удовлетворяющего условию.

```
auto it = std::find_if(accounts.begin(), accounts.end(),
    [](const BankAccount& a) {
        return a.getOwner() == "Иванов";
    });
if (it != accounts.end()) { /* найден */ }
```

5.9. Ввод/вывод и форматирование

- `std::cin >> переменная;` – ввод.
- `std::getline(std::cin, строка);` – ввод строки с пробелами.
- Манипуляторы из `<iomanip>`:
 - `std::setw(n)` – устанавливает ширину поля вывода.
 - `std::left, std::right` – выравнивание.
 - `std::fixed, std::setprecision(p)` – фиксированная точность для вещественных.
 - `std::boolalpha` – вывод `true/false` как строки.

Пример форматированной таблицы:

```
std::cout << std::left << std::setw(20) << "Имя"
    << std::right << std::setw(10) << "Возраст" << '\n';
std::cout << std::left << std::setw(20) << person.getName()
    << std::right << std::setw(10) << person.getAge() << '\n';
```

5.10. Пример работы с датой (без классов)

Можно использовать простую структуру:

```
struct Date {  
    int day { 1 };  
    int month { 1 };  
    int year { 2000 };  
};
```

Для сравнения дат можно написать функцию:

```
bool isLater(const Date& d1, const Date& d2) { /* сравнение годов, месяцев, дней  
*/ }
```

Заключение

Данная лабораторная работа позволяет освоить базовые приёмы объектно-ориентированного программирования в C++, понять важность инкапсуляции, научиться проектировать классы и организовывать взаимодействие между ними. Выполнение задания требует аккуратного подхода к вводу/выводу, использованию контейнеров и алгоритмов стандартной библиотеки. Успешное выполнение работы служит основой для дальнейшего изучения более сложных тем ООП (наследование, полиморфизм, шаблоны).